SoK: The Pitfalls of Deep Reinforcement Learning for Cybersecurity

Shae McFadden[†‡§], Myles Foley[‡*], Elizabeth Bates[‡], Ilias Tsingenopoulos[¶], Sanyam Vyas[‖‡], Vasilios Mavroudis[†], Chris Hicks[‡], Fabio Pierazzi[§]
[†]*King's College London,* [‡]*The Alan Turing Institute,* [§]*University College London,*
[‖]*Cardiff University,* [¶]*KU Leuven,* [*]*Devotion AI Labs*

Abstract

Deep Reinforcement Learning (DRL) has achieved remarkable success in domains requiring sequential decision-making, motivating its application to cybersecurity problems. However, transitioning DRL from laboratory simulations to bespoke cyber environments can introduce numerous issues. This is further exacerbated by the often adversarial, non-stationary, and partially-observable nature of most cybersecurity tasks. In this paper, we identify and systematize 11 methodological pitfalls that frequently occur in DRL for cybersecurity (DRL4SEC) literature across the stages of environment modeling, agent training, performance evaluation, and system deployment. By analyzing 66 significant DRL4SEC papers (2018-2025), we quantify the prevalence of each pitfall and find an average of over five pitfalls per paper. We demonstrate the practical impact of these pitfalls using controlled experiments in (i) autonomous cyber defense, (ii) adversarial malware creation, and (iii) web security testing environments. Finally, we provide actionable recommendations for each pitfall to support the development of more rigorous and deployable DRL-based security systems.

1 Introduction

Deep Reinforcement Learning (DRL) has successfully improved the speed of matrix multiplication [47]; discovered new protein structures [69]; and, achieved superhuman performance at game playing [96, 142]. Through continuous interaction with a DRL environment (often simply referred to as the *environment*), DRL agents learn sequential decision-making policies by optimizing the expected cumulative reward over time [127]. Unsurprisingly, these capabilities have motivated applications in cybersecurity, where systems must make adaptive decisions in dynamic, adversarial environments. DRL applications range from detecting malicious activity [89, 109, 129, 130] to evading detection [9, 27, 112, 131, 132, 136, 140, 147, 151] and from autonomously defending networks [17, 49, 52, 55, 58, 63, 75, 128] to discovering vulnerabilities [8, 32, 42, 43, 45, 51, 79, 82, 92].

However, the transition to cybersecurity introduces fundamental challenges compared to traditional domains where DRL has previously excelled, such as game playing [96], robotics [35], and computer chip design [94]. In cybersecurity, adversaries actively adapt their strategies to evade detection, systems evolve over time, and defenders typically observe limited information about attacker intentions and system state [98]. These characteristics violate core assumptions of Markov Decision Processes (MDPs), the model for DRL learning tasks, that presume fixed transition dynamics and complete state observability [127], which can cause agents to be vulnerable to adversaries [121]. In reality, most security tasks are better characterized as Partially Observable MDPs (POMDPs), where agents must infer hidden dynamics from incomplete knowledge: an *observation* of the system state. Further compounding these challenges are additional task-specific requirements, from information constraints (privacy and access limitations) to practical deployment capabilities (query budgets, real-time constraints). Failing to address these can produce agents that perform well in controlled simulations, however, fail unpredictably and dangerously when deployed in operational environments [101, 122].

In this paper, we systematize 11 recurring methodological pitfalls that emerge when DRL is employed in cybersecurity (DRL4SEC). As shown in Figure 1, we organize these pitfalls across the four stages of agent development: environment modeling (green), agent training (blue), performance evaluation (purple), and system deployment (red). Through systematic analysis of 66 DRL4SEC papers, published between 2018 and 2025, we quantify the prevalence and severity of each pitfall. Our review reveals widespread issues where *every paper contains at least two pitfalls*, with an average of 5.8. Furthermore, across the individual pitfalls, 28.8% of papers fall victim to the least common pitfall (Underlying Assumptions), with the most common being present in 71.2% of papers (Policy Convergence). Through experiments across three DRL4SEC domains, we illustrate how each of the pitfalls can cause performance degradation or produce misleading results. Figure 1 shows the pitfalls and specific case study

domains used for each section in this paper.

Collectively, these pitfalls obscure whether reported improvements stem from genuine algorithmic advances or rather from artifacts of simplified environments and incomplete evaluations. More critically, they create a dangerous gap between simulated and operational reality, leading to systems that provide a false sense of security when deployed. Our goal is not to discourage DRL applications in cybersecurity, as it holds genuine promise, but rather to establish methodological standards that enable the community to distinguish robust, deployable solutions from those that succeed only in idealized settings. In summary, we make the following contributions:

- **Prevalence Analysis** We systematically examine 66 DRL4SEC papers published between 2018 and 2025, quantifying the occurrence and severity of pitfalls across diverse security domains.
- **Pitfall Taxonomy** We define 11 pitfalls when adapting DRL to cybersecurity, organized across four stages of development (modeling, training, evaluation, and deployment).
- **Impact Demonstration** Through experiments using existing DRL4SEC environments in three representative domains, we provide empirical evidence of the impact caused by these pitfalls.
- **Actionable Recommendations** We provide concrete guidance to mitigate each pitfall, supporting the development of more rigorous and reliable DRL4SEC research.

Remark This work identifies recurring pitfalls in DRL4SEC literature. These are not failures of *individual* researchers but *systematic* challenges that emerge when adapting DRL to cybersecurity.

2 Foundational Concepts

At its core, Reinforcement Learning (RL) problems learn optimal policies through environmental interaction formalized as MDPs. MDPs are defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma)$, consisting of: a state space $\mathcal{S}$, action space $\mathcal{A}$, reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, transition function $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ (where $\Delta(\mathcal{S})$ denotes probability distributions over states), and discount factor $\gamma \in [0,1]$ [14, 70, 127]. An RL 'agent' observes a state s_t, selects an action a_t according to the policy π, then receives a reward r_t, and transitions to the next state s_{t+1}. The agent then learns a policy that maximizes expected cumulative discounted reward: $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$. Traditional RL algorithms such as Q-Learning [139] maintain explicit tables of state-action values that do not generalize to unseen states, and become intractable for large state or action spaces, which is critical for cybersecurity applications [99]. DRL addresses this limitation by using deep neural networks as function approximators, allowing generalization to unseen states and scalability to high-dimensional problems [96]. The three primary paradigms of DRL are: *value-based methods*, which approximate the state-action values of Q-learning [96, 134, 138]; *policy-based methods*, which directly approximate the action distribution of the policy [115]; and, *actor-critic methods* that combine both paradigms [95, 116]. The MDP formulation relies on the *Markov property*: that the future state depends *only* on the current state and action, not on the history of past states [29]. This assumption is critical for DRL but can be violated in cybersecurity settings where attackers maintain a hidden state (*e.g.,* compromised credentials, established persistence) not observable to defenders. In such cases, the problem is more accurately modeled as a POMDP, where the agent receives observations o_t from an observation function $\Omega : \mathcal{S} \to O$ rather than directly receiving states [20]. POMDPs require agents to maintain belief states or use recurrent architectures to infer the hidden state from observation histories.

3 Review Methodology

We followed a similar paper collection and review framework as was used in [13, 44], however, as DRL4SEC is still a growing community we extend our search beyond just top-tier conferences (*e.g.,* USENIX Security or IEEE S&P). Therefore, we conducted a systematic literature collection using IEEE Xplore, ACM DL, and Google Scholar. The search keywords can be found in our repositories (See Appendix B).

Collection We performed our collection in two rounds, first in April 2025 using only Google Scholar for initial pitfall identification and second in December 2025 using all search tools to complete the corpus and update citation counts. After the final collection, we performed de-duplication, resulting in 1,603 papers. Then, we performed the *Significance Criteria* filtering to arrive at 363 papers. Finally, we screened the papers for *Topic Criteria* filtering alongside requirements for language (English) and length ($>$ 4 pages) to produce the final 66 papers used in our work, with 37 from the April and 29 from the December collection.

Significance Criteria To ensure academic impact and quality, papers must satisfy at least one of the following criteria: (1) published in a first- or second-tier (CORE-ranked[1] A* or A) venue in machine learning, software engineering, or security; or (2) demonstrate significant research impact by averaging more than 10 citations per year since publication from a reputable publisher (see Appendix B).

Topic Criteria We define the scope of papers to be included in our work as: *direct* applications of DRL to *cybersecurity* problems and have DRL as *the primary methodological contribution*. We then excluded several categories of related but out-of-scope work: (1) secondary studies (surveys, systematic reviews, *etc.*); (2) research on the security *of* DRL sys-

[1] https://portal.core.edu.au/conf-ranks/

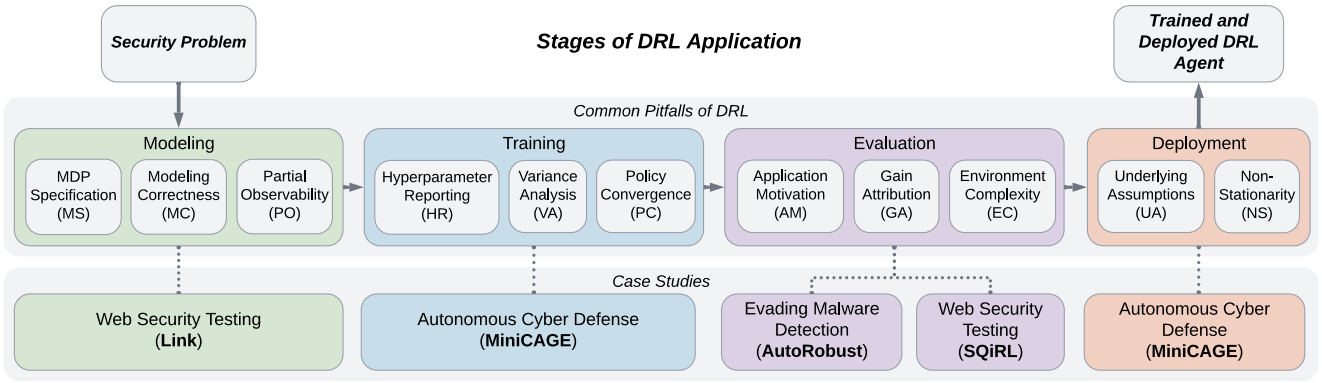


Figure 1: Common pitfalls of DRL applied to cybersecurity, organized by development stage and separate case studies.

tems, *i.e.,* non-DRL-based attacks against DRL agents (which are not applied to *cybersecurity* problems); (3) pure game-theoretic approaches without DRL; (4) non-deep RL methods (*e.g.,* tabular Q-learning, which fail in complex environments as discussed in Section 2); and (5) Multi-Agent RL (MARL) systems, a separate research domain (which require distinct formulations and analysis).

Corpus Characteristics The 66 papers in our corpus were published between 2018 and 2025 across a diverse set of cybersecurity tasks: *adversarial sample creation* (15 papers, 22.7%) [9, 12, 27, 46, 57, 65, 110, 112, 131, 132, 136, 140, 144, 147, 151]; *intrusion detection* (15 papers, 22.7%) [21, 38, 60, 67, 83, 84, 97, 100, 107, 109, 118, 119, 129, 130, 146]; *fuzzing and web security* (10 papers, 15.2%) [8, 19, 32, 42, 43, 45, 51, 79, 82, 113]; *autonomous cyber defense* (7 papers, 10.6%) [17, 55, 58, 73, 75, 128, 149]; *penetration testing* (6 papers, 9.1%) [25, 53, 68, 81, 87, 145]; *cyber-physical systems security* (4 papers, 6.1%) [76, 78, 105, 137]; *malware detection* (4 papers, 6.1%) [7, 15, 24, 37]; *blockchain security* (3 papers, 4.5%) [36, 66, 125]; and *hardware security* (2 papers, 3.0%) [56, 85]. These domains represent the major application areas of DRL4SEC, showing concentration in areas requiring complex sequential decision-making.

Review Process We began with a pilot review of the 37 papers from the April collection to identify themes and commonalities. Using this review, we isolated the reoccurring issues in DRL4SEC papers and formalized the 11 pitfalls discussed in this work. We then evaluated the complete 66 papers for each of the identified pitfalls. Each paper was independently evaluated by *two reviewers* using the pitfall definitions provided in the following Sections 5–8. Reviewers classified each of the 11 pitfalls as *present* (clearly and completely exhibited), *partially present* (partial manifestations or acknowledged but unaddressed), or *not present* (adequately handled or not applicable). Following the independent reviews, the initial prevalence scores achieved substantial agreement, according to a Cohen's κ of 0.712 and the Landis–Koch interpretation [77]. With a 81.5% total agreement on pitfall prevalence, the remaining disagreements (18.5%) around the severity of pitfalls for the papers reviewed were resolved through discussion between the reviewers. When evidence was ambiguous, we gave the benefit of the doubt to the paper by using the less severe pitfall classification to resolve the disagreement (*e.g.,* marking as *not present* if reviews were split between *not present* and *partially present*). The final prevalence scores determined through this process form the basis of our analysis. Further breakdown of agreement, kappa, and interpretation can be found in our repositories (see Appendix B).

For each pitfall, we represent its prevalence using a pie chart alongside its definition with present, partially present, and not present in dark, light, and gray colors, respectively.

4 Case Study Environments

We conducted controlled experiments across three DRL4SEC domains to demonstrate the practical impact of the identified pitfalls. First, we consider Autonomous Cyber defense (ACD) as these environments are well established and often reused across subsequent works [11, 16, 17, 31, 41, 49, 52, 63]. Then, we consider adversarial malware generation and web security testing as they are some of the most prominent applications of DRL4SEC across the 66 papers studied (22.7% and 15.2%, respectively, see Section 3). In this section, we provide a high level overview to the environments used in our case studies; while specific experiment details are provided in their respective sections. For all environments, their full MDP specifications are included in our repositories (see Appendix B).

Autonomous Cyber Defense For our experiments in ACD, we use the MiniCAGE [41] environment, which replicates and extends the network defense task proposed in the CAGE 2 challenge [124], used in a number of subsequent works [11, 16, 17, 49]. The enterprise network to be defended comprises 13 hosts across three subnets: a user subnet (5 hosts), an enterprise subnet (3 hosts plus 1 defender host), and an operational subnet (4 hosts including a critical server). The

defensive (blue) agent must prevent an attacker (red) from reaching the critical server. MiniCAGE contains two attacker strategies: the B-Line attacker proceeds directly through the network to compromise the critical server as quickly as possible, while the Meander attacker attempts to compromise the entire network while en route to the critical server. Using this well established domain we ablate training procedures (Section 6.4) and deployment assumptions (Section 8.3), to show how they can degrade performance.

Adversarial Malware Creation We implement the malware detection and adversarial training environment AutoRobust [132] using publicly available data.[2] In this environment, the agent is presented with a detected malware sample, and must transform it so that it is misclassified. Rather than modifying the executable, the agent operates on a structured behavioral representation extracted from sandbox execution, consisting of system interactions such as file accesses, registry modifications, executed commands, and API usage. At each step, the agent edits or adds report entries to cross the decision boundary while maintaining a plausible behavioral trace. Using this environment, we disentangle the performance contributions of state/action space design from the learned policy, under both fully and partially observable settings. (Section 7.4).

Web Security Testing We consider two payload generation environments: Link [79] for Cross Site Scripting (XSS) and SQIRL [8] for SQL injection. The Link DRL agent uses a hand-crafted state-action space to iteratively build XSS payloads. We retrain Link on WAVSEP, a collection of deliberately vulnerable web-applications, following the settings from the original work. WAVSEP contains 32 XSS vulnerabilities and is used by Lee *et al.* [79]. We show the importance of environment modeling pitfalls by removing: the misaligned and partial observable states (Section 5.4). The SQIRL DRL agent uses embeddings of SQL payloads, and statements to handle a dynamic action space, to generate payloads. We retrain SQIRL on its training benchmark, the SMB, which includes 20 training samples, and 10 testing samples (including 5 reproduced CVEs), following the original training and settings of the work. We show the effect of environment complexity by training SQIRL with and without experience bypassing sanitization; we then test them with both sanitized and unsanitized vulnerabilities (Section 7.4).

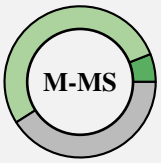
5 Pitfalls of Modeling Environments

The foundation of applying DRL is formulating a problem as an MDP [29, 127]. As a result of problem diversity, cybersecurity domains typically lack standardized DRL environments necessitating bespoke MDP formulations for each task. We identify three pitfalls in environment modeling (shown as green in Figure 1) that pose particular risks for security applications. First, an incomplete *MDP Specification* (Section 5.1) hinders reproducibility and obscures potential flaws in DRL-based approaches. Second, *Modeling Correctness* (Section 5.2) issues arise when the underlying security task is incorrectly modeled as an MDP. Third, security problems often lack complete system information in practice; inadequately addressing this can lead to problems of *Partial Observability* (Section 5.3).

5.1 MDP Specification

The MDP defines the numerical optimization objective (*i.e.,* reward function) and structure of the environment (*i.e.,* state-action-transition dynamics) in which the agent learns from interaction [127]. This is similar to the loss function and data distribution of input-output pairs in supervised learning. An unambiguous MDP definition is thus essential for evaluating the suitability of any implementation or claims regarding agent performance [22]. However, we find that the MDPs specification in the cybersecurity literature often lacks comprehensive definitions. Common ambiguities in MDP descriptions include high-level overviews and missing transition dynamics open to multiple interpretations.

MDP Specification Pitfall
The MDP is ill-defined, so that one or more components (state space, action space, reward function, and transition dynamics) remain unclear.



Prevalence of M-MS Across the 66 papers reviewed, we observed fully unspecified MDP definitions in 4 papers (**6.1%**), partially unspecified definitions in 35 papers (**53.0%**), and complete definitions in the remaining 27 papers (**40.9%**). Among the full and partial cases the distribution of underspecified components are as follows: the state space was the most frequent (22 papers, 33.3%), followed by transitions (18 papers, 27.3%), action space (14 papers, 21.2%), and reward function (10 papers, 15.2%). The high prevalence indicates a significant documentation issue in DRL4SEC literature, reflecting the inherent difficulty of translating complex security scenarios into fully specified environments [17, 39].

Security Implications of M-MS The risks that ambiguous definitions can cause in real-world deployments is threefold. First, it obfuscates other pitfalls in the problem formulation, *e.g.,* state spaces that do not address partial observability allow attacks to go undetected despite causing harm to the system (see Section 5.3). Second, it hinders reproducibility, as attempts at reconstruction may yield different MDPs. For instance, 'rewards for attack detection with penalties for false positives' can produce vastly different behaviors due to variations in relative reward magnitudes [15]. Finally, assessing

[2] https://www.kaggle.com/datasets/greimas/malware-and-goodware-dynamic-analysis-reports

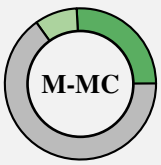
the real-world viability of the proposed solution becomes significantly harder. The lack of implementation details, *e.g.,* an underspecified state, can prevent assurance and auditing of potential attack vectors used in the MDP, thereby obfuscating the attack surface for defenders.

Recommendations for M-MS Complete MDP specification requires unambiguous definition of: (1) the state space, including available information, dimensionality, and semantic meaning of individual state values; (2) the action space, specifying all available actions and connection to the security task; (3) the transition function, describing how actions influence state changes, including stochasticity and terminal conditions; and (4) the reward function, providing the complete formulation with rationale for the contribution of each component to the overall objective, *e.g.,* [8, 9, 113]. Crucially, assumptions, approximations, or simplifications must be explicitly discussed. When space constraints prevent full specification in the main text, this should be provided in appendices or supplementary materials, such as Lee *et al.* [79]. Clear MDP definitions, combined with publicly available code, enable future work to avoid ambiguity, improve reproducibility, and expand on existing approaches.

5.2 Modeling Correctness

Cybersecurity applications typically lack pre-existing environments, requiring bespoke MDP modeling, which can lead to subtle but fundamental errors in problem formulation. As a result, seemingly correct MDPs may violate the Markov assumption or fail to encapsulate the real cybersecurity task. The Markov property (assumed to be true in MDPs) requires that future states and rewards depend only on the *current* state and action [127]. However, this assumption breaks down in two cases: when environments rely on the *entire* history, or when state transitions are *independent* of the current state and action. Even when the Markov property is upheld, M-MC can manifest through misalignment between the modeling of states, actions, or rewards, and the underlying security task. Such misalignment is difficult to detect as policies can appear to perform well by achieving high rewards. For example, a network defense policy that locks all accounts due to misaligned rewards leads to a policy that has no real value.

Modeling Correctness Pitfall
The modeled environment violates the Markov assumption or does not correctly encapsulate the actual MDP underlying the security problem.



Prevalence of M-MC Across the 66 papers reviewed, we observed incorrect MDP modeling in 17 papers (**25.8%**), minor modeling errors in 6 papers (**9.1%**), and valid modeling in the remaining 43 papers (**65.2%**). The combined 34.9% of partially and fully present is primarily due to issues with state representation and transitions. While a general understanding of the MDP framework is prevalent, unaddressed violations of the Markov property are still contained in a third of papers reviewed. This can be attributed to both the lack of standardized environments and the task being poorly suited to being formulated as an MDP.

Security Implications of M-MC When the Markov property is violated or the task is misspecified as an MDP, agents can learn spurious correlations between unrelated decisions. This can result in policies that exploit artificial sequential patterns rather than genuine security relationships. For example, formulating a classification task as sequential decision-making induces nonexistent temporal dependencies between independent samples [89]. Similarly, violations of the Markov property lead to biased and unstable learning, causing agents to exploit artifacts of the formulation rather than genuine structure in the problem. Separately, misaligned rewards can incentivize agents to maximize return through behaviors that undermine security objectives via *reward hacking* [3, 62, 122]. Together, these modeling errors produce brittle policies that appear effective in controlled experiments but fail unpredictably when deployed. Policies learned on an MDP that does not reflect the real-world scenario lead to systems that are not only less effective but potentially more vulnerable.

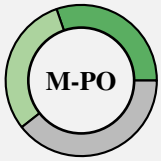
Recommendations for M-MC Modeling of security task as MDPs must consider both theoretical and practical validity of the formulation. First, the state transitions must reflect genuine sequential structure (next states are consequences of current state *and* action) rather than artificially chaining independent decisions. For example, independent samples in a detection task should seek alternative methods such as deep contextual bandits [148, 152] to avoid spurious correlations between states [89]. Secondly, MDP components should be carefully defined such that: rewards provide learning signals to incentivize solving the security task rather than gaming the reward function; actions influence subsequent states to enable rational policy learning; and states include all available information of the environment without conflating details [16, 51, 128]. If complete state information is unavailable in practice, the problem should be formulated as a POMDP (see Section 5.3) with appropriate constraints rather than forcing a potentially misaligned MDP representation.

5.3 Partial Observability

Security problems often inherently lack complete information, for example a defensive agent does not know the full range of attacks an adversary could employ. Such problems are described as *partially observable*. In POMDPs, agents have only partial knowledge of an otherwise 'hidden' state [59]. Treating a POMDP as fully observable when it does not model full system dynamics, can lead to suboptimal behavior and systematic blind spots in the learned policy [121]. Therefore,

when an environment is partially observable, the implications must be explicitly analyzed and appropriately mitigated.

Partial Observability Pitfall
The environment is inherently partially observable (i.e. a POMDP); however, it is treated as if it were fully observable *without* mitigating the partial observability.



Prevalence of M-PO Across the 66 papers reviewed, we identified clear partial observability issues in 20 papers (**30.3%**) and lesser issues in 20 papers (**30.3%**). Of these 40, 8 papers (12.1%) explicitly acknowledge the presence and potential issues of partial observability, yet do not mitigate it. The remaining 26 papers (**39.4%**) either did not consider a partially observable task or directly accounted for it in their modeling. Such prevalence indicates that partial observability is a pervasive challenge in DRL applications to cybersecurity. While the number of papers acknowledging the issue show that some awareness exists, it does not match the high prevalence of partial observability in cybersecurity tasks. Furthermore, this highlights the need to translate solutions from the broader DRL community into cybersecurity contexts.

Security Implications of M-PO Unmitigated partial observability often manifests as a vulnerability *after* training. Consider an agent trained on incomplete information, if partial observability remains unmitigated, the agent may not infer the hidden dynamics of the environment required for effective performance. Therefore, agents can develop 'blind spots' due to the unobservability of a system. Such policies may be optimal in simulation, but in the real-world they have a limited 'key hole' view, leaving them vulnerable to malicious actors.

Recommendations for M-PO Partial observability may be inherent to security tasks which often feature incomplete, delayed, or deliberately concealed information. Addressing this in modeling is therefore essential for making robust decisions under uncertainty. In such cases it should be *explicitly* formulated as a POMDP, as done by Terranova *et al.* [128]. Agents can then use the (partial) information to infer the underlying, hidden state. Commonly this can be achieved by either encoding recent observations into the state representation, such as frame stacking used in Atari games [96], or employing recurrent policies (*e.g.,* LSTM or Transformer) [59, 93] to maintain an internal memory. Other approaches aim to model the unobserved information, using techniques such as Hidden Markov Models [143] or model-based approaches [71], to mitigate the impact of M-PO.

5.4 Case Study of Modeling Pitfalls

We use the Link XSS payload generation environment [79] to show how modeling decisions impact performance. Although the original environment contains valid Markovian transitions, its design introduces unnecessary partial observability and misalignment with the practical security task. This arises from overlapping features in the state space, whereby multiple XSS payloads can be mapped to identical DRL states. For example, a single feature is used to indicate multiple HTML tags (*e.g.,* img, video, audio or svg tags). The unintentional pitfalls introduced by this design can be rectified through reformulation of the MDP. Thus, we consider two formulations of the MDP for this task: (1) *Original*, as published in [79] that have merged states; (2) *Distinct States*, that unrolls the conflated payload features into distinct values, increasing the state dimensionality by 12.8% from 47, to 53. Table 1 shows the performance of these two MDPs over 20 runs.

Table 1: Percent of vulnerabilities found (VF%) and 95% CI results for Link over 20 runs demonstrating the impact of modeling choices.

MDP	Vulnerabilities Found	
	VF (%)	95% CI
Original	75.2	(59.4, 90.9)
Distinct States	85.3	(76.7, 94.0)

Modeling Correctness & Partial Observability Introducing *Distinct States* for each possible payload aligns the MDP with the underlying security task avoiding M-MC. Therefore, the agent can develop correct payloads by selecting actions and observing the *distinct* impact they have on the current payload. Thus, removing the unnecessary partial observability, M-PO, from the environment. This effect can be observed concretely in Table 1 where including *Distinct States* increases the percentage of vulnerabilities found over *Original* by 10.1% and the raising lower bound performance by 17.3%. Therefore, through minor changes in the state space to remove the M-MC and M-PO pitfalls, we can observe significant improvements in performance.

MDP Specification Although originally defined clearly, the performance difference between MDP formulations in Table 1 demonstrates that when components are left unclear, MDP Specification can severely impact final performance.

6 Pitfalls of Training Agents

Unlike supervised learning, where training typically converges given sufficient data, DRL training is inherently more variable and sensitive to numerous factors including: hyperparameter choices, random initialization, sparsity of reward, and the stochastic nature of exploration [4, 61, 104]. We identify three pitfalls in training that can compromise the trustworthiness of DRL-based systems in security. First, *Hyperparameter Reporting* (Section 6.1) issues arise when critical implemen-

tation details necessary for reproduction and validation are omitted. Second, when *Variance Analysis* (Section 6.2) is not performed across multiple training runs, the reliable performance of the proposed approach is in question. Third, when *Policy Convergence* (Section 6.3) is not achieved, due to premature training termination, it often leads to systems with inconsistent and unpredictable behavior when deployed.

6.1 Hyperparameter Reporting

Hyperparameter and architectural choices for DRL agents can have significant impacts on agent training, and final performance, due to the high sensitivity of DRL to these design choices [40, 104]. We find that DRL4SEC research frequently fails to report DRL hyperparameters, including: agent architectures, random seeds, discount factors, and specific DRL algorithm settings (*e.g.,* clipping coefficient for PPO [117]). This information is essential to enable reproducibility and fair comparison between proposed approaches.

Hyperparameter Reporting Pitfall
The hyperparameters governing training and agent architecture of an approach are not reported.

Prevalence of T-HR Across the 66 papers reviewed, we observed the complete absence of hyperparameters in 21 papers (**31.8%**), partial reporting of hyperparameters in 24 papers (**36.4%**), and complete hyperparameter reporting in the remaining 21 papers (**31.8%**). The high rate of missing hyperparameters, is significant given their importance for DRL. This omission fundamentally undermines reproducibility and hinders the advancement of DRL4SEC applications.

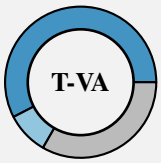
Security Implications of T-HR Since minor variations in design choices can have a disproportionate effect on agent behavior, under-specifying hyperparameters makes results challenging to verify [22, 104]. Hyperparameter transparency is not only a matter of scientific rigor, but a prerequisite for establishing reliable and robust DRL cybersecurity applications that can perform tasks effectively.

Recommendations for T-HR Complete hyperparameter reporting is necessary for reproducible, trustworthy research in cybersecurity as DRL algorithms are highly sensitive to hyperparameter choices [4, 61]. At minimum, publications should report: (1) *algorithm-specific hyperparameters*, *e.g.,* learning rate, discount factor γ, batch size, replay buffer size, and exploration parameters such as ε; (2) *neural network architecture details*, *e.g.,* layer types, dimensions, activation functions, normalization techniques; (3) *optimization settings*, *e.g.,* optimizer type, learning rate, gradient clipping coefficients; and (4) *training procedures*, *e.g.,* total timesteps, update frequency, number of environments for parallel training. When space constraints prevent inclusion in the main text, it should be provided in appendices or supplementary materials, *e.g.,* [21, 128, 131].

6.2 Variance Analysis

Variance in training is an intrinsic property of DRL that can arise from a number of factors including: stochastic environments, exploration noise, and probabilistic transitions [104]. Therefore, training a single policy risks over-interpreting randomness or potential outlier runs as the policy is not representative of aggregate performance. Training multiple policies allows the use of statistical metrics to evaluate the performance and variance of an approach.

Variance Analysis Pitfall
The inherent variance of DRL training is not investigated through explicit analysis of performance over multiple policies.



Prevalence of T-VA Across the 66 papers reviewed, we observed no discussion of multiple training 'runs' or variance analysis in 38 papers (**57.6%**), partial consideration of variance in 6 papers (**9.1%**), and multiple runs with variance analysis in the remaining 22 papers (**33.3%**). Indeed, two thirds of reviewed studies (44 papers, 66.7%) insufficiently consider variance. This lack of analysis obscures genuine methodological advances by reporting results that may not be typical of performance, but instead the result of favorable randomness (*e.g.,* seed values) in training [104].

Security Implications of T-VA Favorable initial conditions, or 'beginner's luck', a single run can over- or underestimate the perceived performance beyond the realistic mean of multiple runs. This is particularly unsafe in security contexts where consistent and reliable performance is crucial. Dismissing an approach based on a single bad run that can perform well *on average*, risks rejecting a valuable security solution. Alternately, agents that over-promise and under-deliver on average, expose end users to unnecessary risk.

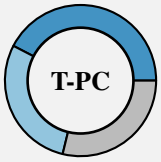
Recommendations for T-VA As a lone trained agent is insufficient to establish representative performance of that agent [5], it is essential to train and evaluate multiple trained policies and report the statistical variance. Yet, there is no canonical number of policy runs N to use for evaluation. While it has been argued that $N \geq 20$ is a reasonable mean [6, 30], complex environments may require $N \geq 50$ for robust 95% confidence intervals [5], and often DRL papers report $N \leq 5$ [28, 34, 103, 117]. Consequently, we recommend a minimum of $N \geq 5$, and encourage higher runs ($N \geq 20$ or $N \geq 50$) for stronger statistical claims [104]. To better represent the variance in agent behavior, statistical metrics must

also be reported [5, 23, 104], for example: 95% confidence intervals, inter-quartile range (IQR), and risk-sensitive measures such as Conditional Value at Risk (CVaR). Variance should be reported within and between training episodes, and certainly for the final policies during testing [23]. Examples of such testing can be seen in [17, 36, 43]. In cybersecurity, reliability directly impacts security posture. Transparent variance reporting enables informed risk assessments from practitioners, preventing the reliance on potentially misleading results that will not reflect performance in deployment.

6.3 Policy Convergence

Training in DRL should result in a policy that has converged to a *stable* strategy, even if sub-optimal. Determining convergence is a difficult and open problem in DRL [28, 103, 117]. However, in DRL4SEC literature the training length, a key aspect in investigating convergence, is either frequently not reported or unjustified. Without presenting reward or performance metrics during training, it is impossible to determine policy convergence or premature training termination.

Policy Convergence Pitfall
The convergence of the policy is not demonstrated prior to the termination of training.



Prevalence of T-PC Across the 66 papers reviewed, we observed no demonstration of convergence in 28 papers (**42.4%**), partial demonstration or discussion in 19 papers (**28.8%**), and demonstration of convergence in the remaining 19 papers (**28.8%**). While many papers discuss the reasoning for the length of training used or mention convergence, explicit demonstration is often missing. The gap in convergence reporting suggests that although the need for convergence may be understood, the concrete demonstration of policy convergence and stability is often under-emphasized. In this way, the reliability of the learned behaviors is uncertain.

Security Implications of T-PC Without demonstrating convergence, it is difficult to establish if an agent has learned a truly robust policy or if premature termination has left gaps in its decision-making for certain scenarios [141]. Non-converged policies pose an increased risk as they behave unpredictably, which can be exacerbated in rare edge cases [126]. In such scenarios, even otherwise optimal agents can behave erratically, leading to clear vulnerabilities in deployment [54, 126].

Recommendations for T-PC The convergence of policies should be evaluated over the learning process to reveal training dynamics, such as instability or temporary performance plateaus before final convergence. We recommend the inclusion of figures showing episodic reward or task-specific metrics as clear evidence of stabilization before training termination [86, 104]. To ensure convergence is consistent, measures of training variance should be included as discussed in Section 6.2. This approach aids in the assessment of policies that represent genuinely converged behaviors, and not merely snapshots of incomplete learning that may fail unpredictably, as can be seen in [82, 85, 137].

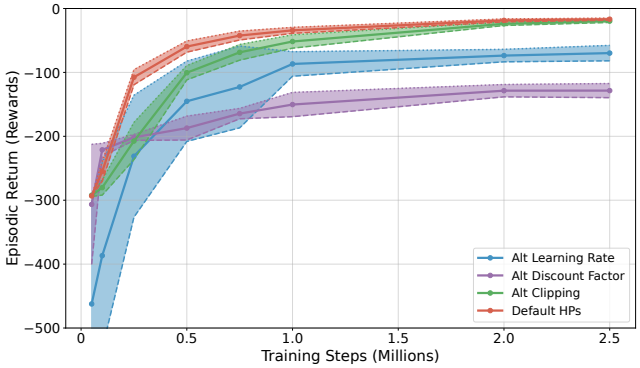


Figure 2: Training performance of different hyperparameter sets, across 20 runs using 95% CI for variance. Zero is the theoretical max score in MiniCAGE.

6.4 Case Study of Training Pitfalls

To evaluate the impact of pitfalls in policy training, we use MiniCAGE [41], described in Section 4. Specifically, we evaluate four hyperparameter settings for T-HR, investigate variance and convergence for T-VA and T-PC, respectively.

Impact of Hyperparameters In Figure 2, we show the performance of a default hyperparameter set with three modifications: 'Default HPs' (red) represents the default hyperparameters in Stable Baselines 3[3] for PPO; 'Alt Learning Rate' (blue) modifies the learning rate; 'Alt Discount Factor' (purple) modifies the discount factor for future rewards; and 'Alt Clipping' (green) modifies PPO's loss clipping. We examine the final performance at 2.5 million steps and see how changing a single hyperparameter results in drastically different performance, highlighting how hyperparameter transparency is needed to ensure reproducibility.

Analyzing Variance For each evaluation in Figure 2, we plot both the mean performance of twenty agents and the 95% Confidence Interval (CI) upper and lower bounds. We can see the potential impact of not performing variance analysis. Concretely, at 500k the upper CI bound of the 'Alt Learning Rate' (blue) appears to be the second best performing agent. However, we see from the variance that an agent at this upper CI bound is not representative of the (lower) mean performance. By evaluating over multiple runs and reporting 95% CI, we obtain a more accurate picture of actual performance.

[3] https://github.com/DLR-RM/stable-baselines3

Demonstrating Policy Convergence Figure 2 also plots the policy performance (episodic rewards) over training. The 'Default HPs' (red) and 'Alt Clipping' (green) settings show relative convergence at 2.0-2.5 million steps where plateauing performance curves and minor variance can be seen. In contrast, the 'Alt Learning Rate' (blue) and 'Alt Discount Factor' (purple) are still increasing in performance with higher variance at 2.5 million steps. Plots such as this, in conjunction with variance analysis, can be used to effectively demonstrate convergence and motivate training length chosen.

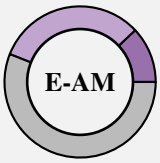
7 Pitfalls of Evaluating Agents

DRL evaluations present unique challenges compared to traditional machine learning. We identify three pitfalls in agent evaluation that can undermine DRL-based approaches in cybersecurity. First, *Application Motivation* (Section 7.1) issues occur when DRL is used without adequate justification over simpler domain-specific methods. Second, *Gain Attribution* (Section 7.2) issues arise when performance gains are due to the basic agent capabilities rather than the learned policy itself. Third, *Environment Complexity* (Section 7.3) creates misleading results when training and evaluation occur in overly simplified settings that do not capture real-world complexity.

7.1 Application Motivation

Motivating an applied approach and comparing to established baselines are standard scientific practice. Therefore, the application of DRL must be motivated over existing approaches both empirically (through experimental comparison) and theoretically (by justifying the value of DRL-based approaches). Yet, direct and fair comparisons to existing approaches in the domain can be infeasible when reformulating through a DRL paradigm. For example, Goel *et al.* [55] design *dynamic* active directory defense as a two player attacker-defender game, as opposed to static detection, thereby making comparison with existing approaches unreasonable. It is therefore crucial in such cases to theoretically motivate the necessity of DRL over existing formulations (for example see Section 7.4).

Application Motivation Pitfall
The application of DRL over traditional methods is not justified empirically or theoretically.



Prevalence of E-AM Across the 66 papers reviewed, we found that 8 papers (**12.1%**) provide neither empirical or theoretical motivation for their use of DRL; 21 papers (**31.8%**) offered only one of the forms of motivation; and 37 papers (**56.1%**) motivated in both forms. The majority of partial justification (17 papers, 25.8%) arises from only having theoretical motivation. The absence of appropriate baselines may stem from the difficulty of aligning reformulated tasks to prior work, or a focus shift from investigating concrete improvements to the novelty of applying DRL.

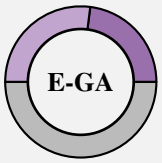
Security Implications of E-AM The application of DRL in security is relatively under-explored in comparison to other learning paradigms. Replacing extensively studied methods with DRL, besides the other pitfalls we explore here, runs the risk of introducing novel attack surfaces [54, 120, 126]. Therefore, the benefits of a DRL-based approach should surpass engineering in-domain approaches to replicate the sequential nature of DRL environments.

Recommendations for E-AM While DRL can perform a wide variety of complex tasks, much like other ML paradigms it is not suitable for all tasks. As a result, applying DRL to a novel task should be justified theoretically and, when possible, empirically. When baselines are available for comparison, they should be used to demonstrate the advantage of DRL, in addition to the motivation for applying DRL in the first place [27, 130, 144]. When baselines are not available, motivation *must* demonstrate the *necessity* of re-formulating to a DRL task [75, 128].

7.2 Gain Attribution

As discussed in Section 5.1 framing a security task as an MDP is the foundation of applying DRL. However, engineering a functional environment entails additional design choices beyond state representation, particularly in how actions are translated into concrete effects in the cybersecurity task [104]. These artifacts can introduce performance gains that overstate the actual contribution of the learned agent.

Gain Attribution Pitfall
Improvements stem from additional information or capabilities provided in MDP design rather than the learned policy itself.



Prevalence of E-GA Across the 66 papers reviewed, we found that 15 papers (**22.7%**) did not clearly demonstrate that the agent improved over baseline environmental performance; 18 papers (**27.3%**) demonstrated partial evaluation of where performance gains come from; and 33 papers (**50.0%**) adequately showed the contribution of the learned policy. Notably, 6 of the 15 papers fully exhibiting this pitfall *explicitly* acknowledge it but do not attempt to disentangle the sources of improvement.

Security Implications of E-GA Misrepresentative gains can create false confidence in a proposed DRL approach. Even when an approach outperforms state-of-the-art, gains may stem primarily from baseline environmental capabilities

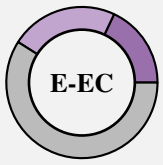
rather than the learned policy, undermining conclusions about the necessity and benefit of DRL [61]. Such agents can give a false sense of robustness as their learned policy is not the primary source of effectiveness, making it easier for an adversary to attack [54, 141]. Thus such improvements reflect stronger baseline design rather than effective learning, and could be replicated by other agents given the same capabilities.

Recommendations for E-GA The value of the learned policy can be disentangled from environmental artifacts through two kinds of ablation. First, evaluating performance under random action sampling isolates the contribution of policy learning and establishes a baseline performance in the environment [104]. As a result, improvements that arise solely from state-action engineering should be reported separately from the trained policy performance, for example: [19, 81]. Second, comparisons between different approaches (both DRL and other-paradigms) should be performed with equivalent information and capabilities (*e.g.,* state–action spaces), as shown in [37, 137]. This isolates the improvements introduced by MDP engineering, such as expanded information in the states and enhanced capabilities of the actions.

7.3 Environment Complexity

Environments provide an interactive interface for a DRL agent, translating agent actions to concrete security outcomes, and providing meaningful feedback, for example mutating payloads and observing their effects [8, 50, 79, 92]. In practice, implementing real-world environments is often complex and highly variable. The same security problem can be approached in a number of ways [1, 11, 17, 31, 41], using environmental abstractions, simulations, or simplifications [1, 43, 76, 137]. Many cybersecurity environments do not demonstrate a strong real-world correspondence and, consequently, agent performance has limited relevance to the fundamental security task [39, 98].

Environment Complexity Pitfall
Training and evaluation occur in an environment that is contrived or overly simplified.



Prevalence of E-EC Across the 66 papers reviewed, we found that 12 papers (**18.2%**) evaluated their approaches in overly simplified or contrived environments; 15 papers (**22.7%**) made some effort to improve environmental realism; and 39 papers (**59.1%**) evaluated on sufficiently realistic environments. The 40.7% of papers with this pitfall highlight that implementing real-world environments is non-trivial.

Security Implications of E-EC Agents trained in overly simplified environments may learn effective policies, but these

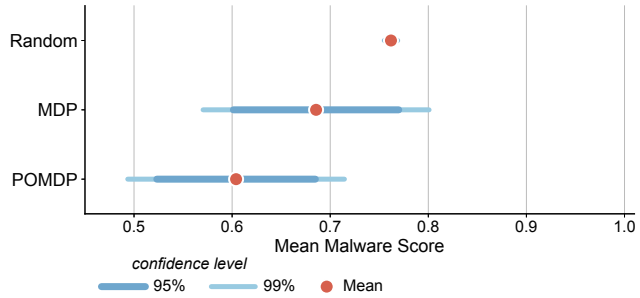


Figure 3: Comparison of mean with 95/99% CI performance for trained and random policies in AutoRobust across 20 runs with 20 evaluation episodes each.

may be incapable of transferring to, or learning in, real environments [62, 122]. Indeed, the transfer from simulated to real environments (commonly referred to as the 'sim-to-real' gap [102]) can lead to catastrophic failures. A concrete example of this is *reward hacking* where agents learn to exploit flaws in environmental modeling to maximize rewards, leading to policies that appear optimal but provide no genuine security value [101, 122]. These flaws become particularly dangerous in security contexts where the stakes of failure are high: organizations may deploy systems based on promising laboratory results only to discover that they underperform or are vulnerable in the real-world [150].

Recommendations for E-EC As a result of transferability issues from the sim-to-real gap, it is best practice to train and evaluate DRL in the most realistic environment feasible. Applying DRL to security problems should follow the evaluation best practices from that specific security domain, *e.g.,*, [9, 19]. For instance, in security domains where real-world data or systems are used (*e.g.,* web security testing often using live applications), DRL should not preclude such an evaluation. However, in some domains the use of real-world data or systems is challenging or infeasible. In such cases benchmarks should be used to demonstrate impact. For example, ACD research relies on a handful of benchmark simulators [1, 2, 10], while fuzzing research recommends benchmarks containing known vulnerabilities [74, 114].

7.4 Case Study of Evaluation Pitfalls

Here we consider the three pitfalls we identified in evaluating agents: first, the attribution of gains in the AutoRobust environment; second, the theoretical motivation of DRL in adversarial malware generation; third, environment complexity comparisons in the SQIRL environment for SQL injection.

Identifying the Source of Gains. To assess the impact of the E-GA pitfall, the goal in the AutoRobust environment (modeled after [132]) is to disentangle the contributions of different elements in the approach; see the results in Figure 3. First, we train and evaluate policies with full access to the

Table 2: Percent of vulnerabilities found (VF%) and 95% CI for SQiRL across 20 runs and evaluations each setting. Showing the impact of evaluating on low-complexity environments.

Training Environment	Evaluation Environment			
	Sanitized		Non-Sanitized	
	VF (%)	95% CI	VF (%)	95% CI
Combined	41.10	(33.9, 48.4)	88.5	(85.4, 91.6)
Non-Sanitized	29.80	(22.9, 36.8)	88.7	(85.5, 91.9)

internal state of the malware detection model, referred to as *MDP*. Second, we consider policies without internal state access, named *POMDP*, to evaluate the performance of learning under a more restrictive and realistic threat model. Finally, we evaluate *Random* policies that sample actions at random (as recommended in Section 7.2), to isolate the intrinsic efficacy of the action space itself. There are two key takeaways from results in Figure 3. First, full observability does not guarantee better performance, as agents can struggle to learn the task even with fully observable information. Second, while trained policies (MDP & POMDP) have the *potential* to achieve better performance, the random baseline is already strong. This demonstrates the importance of E-GA as the action space alone provides a large portion of overall performance.

Motivating the Application. As an example of theoretical motivation, in the domain of adversarial malware creation, DRL has several key capabilities over existing gradient-based attacks. The increasing complexity of AI-based systems makes gradient-based methods intractable. Concretely, gradient-based methods assume a fixed input size, limiting exploration to only a very small subset of the actual transformations an adversary can make [153]. Similarly in malware, gradient-based perturbations on the feature representation do not have a consistent and reliable method for being mapped to valid changes in the problem-space, *i.e.,* the actual program. In contrast, DRL can operate *directly* on malware samples, bypassing the need for feature to problem-space mapping. Furthermore, given an adversary with a well defined sets of actions, at convergence DRL can additionally confer probabilistic guarantees on the level of vulnerability/robustness [132].

Changing Environment Complexity To show the effect of E-EC we use the SMB benchmark from SQIRL [8]. Using the SMB we train two policies, one on the default SMB with a *Combined* sanitized and non-sanitized vulnerability set, and one policy with the same vulnerabilities without sanitization (*Non-Sanitized*). We evaluate both trained policies on the SMB test set of sanitized and non-sanitized vulnerabilities; Table 2 shows the percentage of vulnerabilities found across 20 runs with 95% CI. We see both the *Combined* and *Non-Sanitized* agents are capable of discovering vulnerabilities in the simplified non-sanitized settings. However, when changing the evaluation set to the more challenging *Sanitized* vulnerabilities we see a 47.4% drop in vulnerabilities found for the *Combined* agent, with a further 11.3% for the *non-sanitized* agent. This demonstrates how simplified evaluation environments can inflate performance, producing promising results that fail to translate into real-world capabilities.

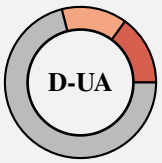
8 Pitfalls of Deploying Agents

Avoiding pitfalls in modeling, training, and evaluation does not guarantee an agent's reliable performance in the complexities and uncertainties of real-world systems. Deployment introduces additional challenges that test the practical feasibility, robustness, and adaptability of DRL-based approaches. We identify two pitfalls in agent deployment that hinder real-world performance. First, *Underlying Assumptions* (Section 8.1) issues arise when agent design or environment assumptions cannot be satisfied in practice. Second, *Non-Stationarity* (Section 8.2) problems occur when the evolving nature of security environments are not adequately captured.

8.1 Underlying Assumptions

Modeling and implementing a security task with DRL can introduce unrealistic assumptions that violate practical deployment constraints. While similar to Modeling Correctness, an environment suffering from D-UA can be theoretically valid, but invalid in practice. For example, assuming perfect knowledge of the network state in ACD, unlimited query budgets for black-box attacks, unrealistic attack capabilities [108], or continued access to ground-truth for rewards in evaluation. While these violations make a problem tractable for research, they can make the proposed approach infeasible, impractical, or unusable in realistic security deployments.

Underlying Assumptions Pitfall
The environment or agent training requirements cannot be satisfied in real-world applications.



Prevalence of D-UA Across the 66 papers reviewed, we found that 10 papers (**15.2%**) contained unrealistic assumptions that would prevent real-world deployment (*e.g.,* network defense on abstract graph structures); 9 papers (**13.6%**) suffered from minor or partially unrealistic assumptions (*e.g.,* requiring real-time high quality labeled data in live deployments); and 47 papers (**71.2%**) operated under realistic deployment assumptions. Across the 19 papers beset by this pitfall, 6 papers (9.1%) acknowledged the unrealistic nature of their assumptions and an awareness of practical deployment challenges. We observe that assumptions regarding information access and capabilities are commonly relaxed when trans-

lating security problems to DRL. Importantly, if constraint relaxation is unavoidable, a clear discussion should be provided to understand the limitations and trade-offs introduced.

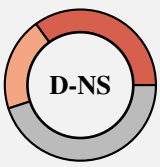
Security Implications of D-UA Underlying Assumptions can create overconfidence in the capabilities of a DRL-based approach, which may not be directly evident during deployment [98]. Such a system could be ineffective, underperforming, or even exploited, due to the *disconnect* between research and actual deployment [54, 101]. Furthermore, in cases of hard constraints like privileged data access or strict query budgets, the whole approach might be rendered invalid.

Recommendations for D-UA To ensure practical relevance and transparency we recommend designing evaluations to mirror real-world problems. However, training effective DRL agents often requires training over *millions* of timesteps and data points [18, 142]. In some cases, sample-efficient or offline methods could minimize the number of data points required. Equally, other cases may require *reasonable* assumptions regarding agent knowledge, capabilities, and operational conditions, including the information and resources that are available [130, 136]. Realistic deployment settings should be used by constraining observation/action spaces, and enforcing practical limits on query budgets and computation. In cases where such assumptions are unavoidable, ablations should show performance and robustness under constrained and noisy conditions [78, 149]. Robustness and generality of results can be further strengthened through adversarial stress-testing to simulate operational uncertainty and attacker adaptation [132]. Finally, every study should include a brief summary of deployment requirements, such as privileges and training budgets.

8.2 Non-Stationarity

DRL assumes static environment dynamics over time, violation of this assumption can lead to poor agent performance and generalization in deployment. Unfortunately, many security tasks are inherently non-stationary [72, 106], including: evolving defenders [26, 90, 91, 111], adaptive adversaries [48, 133], changing network topologies, updated software systems, and shifting attack methodologies. Thus, it is critical that such non-stationary components are integrated into the dynamics of the environment during training.

Non-Stationarity Pitfall The environment does not effectively mitigate the inherent non-stationarity of the underlying cybersecurity task.



Prevalence of D-NS Across the 66 papers reviewed, we found that 23 papers (**34.9%**) failed to mitigate the inherent non-stationarity of their cybersecurity tasks; 13 papers (**19.7%**) considered some of the non-stationarity present in the task; and 30 papers (**45.5%**) did not have, or adequately handled, the non-stationary aspects of their chosen task. The most common unaddressed non-stationarity across the papers was the behavior of other actors in the environment, *e.g.,* assumed fixed attacker in ACD. Not accounting for the continual change in many cybersecurity tasks diminishes the potential for, and evaluation of, long term agent performance.

Security Implications of D-NS In practice, adversaries, network conditions, and software systems evolve continuously; yet agents trained under stationary assumptions learn on fixed dynamics and predictable behavior. Such agents are brittle to change and exploitable in dynamic environments, where they can fail by adopting obsolete strategies, or be manipulated by adversarial tactics. Beyond adversarial evolution, broader concept drift (*e.g.,* firewall or software updates) can degrade performance over time, causing silent failures undermining long-term performance. Critically, systems may appear effective under static evaluation providing a false sense of robustness, but perform unreliably, once deployed in the ever-changing landscape of cybersecurity operations.

Recommendations for D-NS Cybersecurity tasks are inherently dynamic, so DRL agents must be explicitly exposed to such variability during training. Environments should be designed for adaptation; for example, non-stationarity can be incorporated through temporal shifts, randomized parameters, evolving topologies, and adaptive adversaries *e.g.,* [151]. Additionally, techniques such as curriculum learning, domain randomization, and population-based training can further help agents generalize to progressively hardening conditions. Furthermore, adversarial training should be used where appropriate, having both sides evolve iteratively (alternating or concurrently updating attacker and defender policies), *e.g.,* [33, 55]. Training agents against adaptive counterparts simulates the attacker-defender arms race, enabling agents to adapt [133]. Brittleness, and long-term resilience can then be measured through transfer tests against unseen strategies. The intentional embedding of non-stationarity in environments means DRL agents understand the evolving security landscape.

8.3 Case Study of Deployment Pitfalls

We evaluate the impact of pitfalls in deploying DRL agents using MiniCAGE [41] (introduced in Sections 4 and 6.4). First, we consider Underlying Assumptions by modifying the assumed fixed action order between attacker and defender during testing, blue then red (B→R) in the original formulation. Second, we study Non-Stationarity by changing the fixed attacker strategy between training and testing.

Breakdown of Underlying Assumption The first section of Table 3 shows the average reward of the DRL defender when varying the action order of red and blue agents in training and testing (*e.g.,* R→B indicates red then blue and *Mixed*

Table 3: Experimental results for MiniCAGE deploying agents case study. The top section shows blue agent performance when training and evaluation action orders differ. The bottom section shows blue agent performance against different red adversaries. Zero is the theoretical max score.

Training Order	Evaluation Order					
	R→B		B→R		Mixed	
	Score	95% CI	Score	95% CI	Score	95% CI
R→B	**-15.8**	(-14.8, -16.7)	-21.0	(-19.0, -23.1)	**-18.3**	(-17.0, -19.6)
B→R	-72.7	(-65.6, -79.9)	**-17.2**	(-16.7, -17.7)	-45.5	(-42.0, -49.0)
Mixed	-28.4	(-25.6, -31.2)	-19.9	(-19.2, -20.6)	-24.6	(-22.9, -26.2)
Training Red Agent	**Evaluation Adversary**					
	B-line		Meander		Mixed	
	Score	95% CI	Score	95% CI	Score	95% CI
B-line	**-19.0**	(-20.9, -17.2)	-104.1	(-120.7, -87.6)	-61.4	(-70.5, -52.3)
Meander	-79.0	(-97.2, -60.9)	**-61.0**	(-74.7, -47.3)	-70.2	(-83.5, -56.9)
Mixed	-40.2	(-47.1, -33.4)	-79.2	(-86.5, -71.9)	**-60.2**	(-65.8, -54.7)

indicates random turn order). As expected, agents perform comparably when tested on the same training order (-17.2 for B→R and -15.8 for R→B). When the assumed order changes, performance degrades: minimally for R→B agent (-21.0) and drastically for the B→R agent (-72.7). These results highlight how minor design choices can degrade performance if the assumptions they rely on do not hold in practice.

Changing Attacker Strategy Table 3 shows the average reward of defender agents under varying attacker strategies: B-Line and Meander (see Section 4). The agent order is kept constant as B→R. We see a degradation in performance when any stationary blue agent is evaluated against unseen or mixed attacker strategies. This degradation can be mitigated by including non-stationary attacker strategies during training as the Mixed blue agent outperforms the stationary defenders in both mixed and cross evaluations. These results show that policies can be brittle to changing conditions when trained without accounting for non-stationarity.

9 Limitations

By selecting a representative set of academic works, we aim to show that the pitfalls identified are present even in highly-cited or top conference papers. Thus, while extending our analysis to the entirety of DRL4SEC literature would improve the coverage of pitfall prevalence, it would provide limited insight beyond the findings of this work. To minimize bias in reviewing we (a) were lenient in cases of ambiguity, (b) employed a two-reviewer process with 81.5% initial agreement, and (c) resolved disagreements through discussion. To highlight pitfalls we performed didactic case studies, these are intended as illustrative examples where pitfalls have measurable, concrete, consequences in security environments. The specific consequence of a pitfall depends on the security problem, how it is modeled, implemented, algorithm chosen, and numerous other factors. We encourage researchers to carefully consider the pitfalls of DRL for cybersecurity and, where applicable, apply the recommendations from this work.

10 Related Work

To the best of our knowledge, ours is the first work to delineate the pitfalls that arise when applying DRL in security. Many papers, such as Sommer and Paxson [123] discuss use of ML in *different* cybersecurity tasks; we focus our discussion on works identifying the pitfalls in applying ML *paradigms* across cybersecurity. In 2022, Arp *et al.* [13] identified 10 pitfalls in 30 supervised learning-based security research papers, finding at least three pitfalls in each paper. More recently, Evertz *et al.* [44] extended this analysis to LLM-based security research. They identified nine pitfalls, finding at least one in each of the 72 papers reviewed. While there are some parallels between previously identified pitfalls and our work, the sequential decision-making paradigm of DRL is different. This means that although similar in spirit, these parallel ideas manifest in fundamentally different ways. For example, invalid environment modeling opposed to incorrect dataset construction: they have different causes, impacts, and mitigations. Complementing these pitfall-focused papers, several papers have surveyed the application of DRL to specific cybersecurity tasks [64, 88, 99, 135], or security of DRL [80]. Although these studies provide insights in their respective paradigm and application areas, there remains a gap in systematizing the pitfalls specific to DRL for cybersecurity. DRL introduces fundamentally different challenges: from the sequential nature of decision-making to the reliance on environment design [16, 17]. Therefore, by identifying the major shortcomings in current DRL4SEC literature and providing actionable recommendations, we lay the groundwork for future high-quality research.

11 Conclusion

The autonomous and adaptive decision-making that DRL can offer in complex environments has significant potential for cybersecurity applications. However, our review of 66 DRL4SEC papers published between 2018-2025 reveals 11 common methodological challenges (pitfalls) that undermine DRL4SEC research. Most commonly, we find that: 71.2% lack clear evidence of policy convergence, 60.6% fail to address partial observability, 66.7% neglect variance analysis, and 40.9% evaluate in oversimplified environments. Through case studies in ACD, adversarial malware creation, and web security testing, we demonstrate these pitfalls directly impact performance and create potentially brittle policies. Through the identification of 11 DRL4SEC pitfalls and recommendations for their mitigation, this work establishes higher methodological standards for future applications of DRL to cybersecurity.

Acknowledgments

This research was partially funded and supported by: the UK National Cyber Security Centre (NCSC); UK EPSRC Grant no. EP/X015971/2; the Defence Science and Technology Laboratory (DSTL), an executive agency of the UK Ministry of Defence, supporting the Autonomous Resilient Cyber Defence (ARCD) project within the DSTL Cyber Defence Enhancement programme; a Google Academic Research Award (GARA); the Research Fund KU Leuven; the Cybersecurity Research Program Flanders.

A Ethical Considerations

This work identifies common methodological pitfalls in published research. We emphasize that these pitfalls represent *systematic* challenges in adapting DRL to cybersecurity rather than failures of individual researchers. Our goal is constructive: to provide concrete, actionable, recommendations that strengthen future research. We frame our analysis around pitfall patterns over critiquing specific papers, and while we list the papers reviewed, *we do not report the per-paper pitfalls*.

Fairness in review Our review examined 66 papers published between 2018 and 2025. To ensure fairness, each paper was independently evaluated by two reviewers using predefined pitfall definitions, with disagreements resolved through discussion and the benefit of the doubt given when pitfall presence was ambiguous (see Section 3 and Appendix B).

Stakeholders and potential harms Our paper could theoretically affect several stakeholders. First, this work sets higher standards for *researchers*, however, may be perceived as criticism; we mitigate this by reporting only aggregate prevalence and by framing pitfalls as community-wide problems. Second, by explicitly describing the problems in DRL4SEC literature and by providing recommendations we aim to inform *practitioners* to mitigate these failure modes in practice. Third, our discussion could inform *adversaries* to potential weaknesses in DRL-based defenses; however, these vulnerabilities are already implicit in existing publications and, therefore, improving DRL4SEC research provides more value to the security community than it does to potential attackers. Finally, other stakeholders, including organizations that depend on DRL-based tools, receive indirect gains from improvements in DRL4SEC research.

Ethics Decision We conclude that this research is ethical because: (1) it provides substantial benefit to the security research community by identifying and addressing systematic methodological challenges; (2) it does not expose individuals to harm or violate their rights; (3) it treats analyzed prior work fairly and constructively; and (4) the potential for misuse is minimal compared to the benefits of improving research methodology. The decision to publish serves the public interest by advancing the quality and reliability of DRL-based cybersecurity research.

B Open Science

To support open science and promote reproducibility, we provide the following repositories. **GitHub**: https://github.com/alan-turing-institute/DRL4Sec-Pitfalls **Zenodo**: https://doi.org/10.5281/zenodo.20209122

Case Study Environments We provide the version of the environment used to create the results in this paper. This includes: training and evaluation scripts alongside any changes made to the environment to ablate the impact of pitfalls. The complete list of environments is: MiniCAGE ACD environment (Sections 6.4, 8.3), Link XSS environment (Section 5.4), SQIRL SQL Injection environment (Section 7.4), and the AutoRobust adversarial ML environment (Section 7.4). Alongside the code we also report the MDPs and hyperparameters for each environment in repositories.

Literature Review Data The following information from our literature review can be found as follows: (1) a complete list of the 66 papers analyzed with bibliographic information (reported through corpus characteristics, Section 3, and associated references); (2) the search keywords (reported in our repositories); and, (3) the inclusion/exclusion criteria used (reported in Section 3 and our repositories). We have chosen to not release the per-paper pitfalls, similar to [13, 44], in order to keep the constructive nature of the paper.

References

[1] Cyberbattlesim. https://github.com/microsoft/cyberbattlesim, 2021.

[2] Cage challenge 2. https://github.com/cage-challenge/cage-challenge-2, 2022.

[3] Sahar Abdelnabi et al. Measuring security without fooling ourselves: Why benchmarking agents is hard. *arXiv:2605.22568*, 2026.

[4] J. Adkins, M. Bowling, and A. White. A method for evaluating hyperparameter sensitivity in reinforcement learning. *Advances in Neural Information Processing Systems*, 2024.

[5] R. Agarwal et al. Deep reinforcement learning at the edge of the statistical precipice. *CoRR*, 2021.

[6] R. Agarwal et al. Reincarnating reinforcement learning: Reusing prior computation to accelerate progress. *Advances in Neural Information Processing Systems*, 2022.

[7] M. Al-Fawa'reh, J. Abu-Khalaf, P. Szewczyk, and James Jin Kang. Malbot-drl: Malware botnet detection using deep reinforcement learning in iot networks. *IEEE Internet of Things Journal*, 2023.

[8] S. Al Wahaibi, M. Foley, and S. Maffeis. SQIRL: Grey-box detection of sql injection vulnerabilities using reinforcement learning. In *USENIX Security Symposium*, 2023.

[9] Hyrum S Anderson, A. Kharkar, B. Filar, D. Evans, and P. Roth. Learning to evade static pe machine learning malware models via reinforcement learning. *arXiv:1801.08917*, 2018.

[10] A. Andrew, S. Spillard, J. Collyer, and N. Dhir. Developing optimal causal cyber-defence agents via cyber security simulation. In *Workshop on Machine Learning for Cybersecurity*, 2022.

[11] A. Applebaum et al. Bridging Automated to Autonomous Cyber Defense: Foundational Analysis of Tabular Q-Learning. In *ACM Workshop on Artificial Intelligence and Security*, AISec'22, 2022.

[12] G. Apruzzese, M. Andreolini, M. Marchetti, A. Venturi, and M. Colajanni. Deep reinforcement adversarial learning against botnet evasion attacks. *IEEE Transactions on Network and Service Management*, 2020.

[13] D. Arp et al. Dos and Don'ts of Machine Learning in Computer Security. In *Proc. of the USENIX Security Symposium*, 2022.

[14] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 2017.

[15] A. S. Basnet, M. C. Ghanem, D. Dunsin, H. Kheddar, and W. Sowinski-Mydlarz. Advanced persistent threats (apt) attribution using deep reinforcement learning. *Digital Threats: Research and Practice*, 2025.

[16] E. Bates, C. Hicks, and V. Mavroudis. Beyond rewards in reinforcement learning for cyber defence. In *International Conference on Machine Learning*, 2026. To appear.

[17] E. Bates, V. Mavroudis, and C. Hicks. Reward shaping for happier autonomous cyber security agents. In *ACM Workshop on Artificial Intelligence and Security*, 2023.

[18] C. et al. Berner. Dota 2 with large scale deep reinforcement learning. *arXiv:1912.06680*, 2019.

[19] K. Böttinger, P. Godefroid, and R. Singh. Deep reinforcement fuzzing. In *IEEE Security and Privacy Workshops*, 2018.

[20] C. Boutilier, T. Dean, and S. Hanks. Decision-theoretic planning: Structural assumptions and computational leverage. *Journal of Artificial Intelligence Research*, 1999.

[21] G. Caminero, M. Lopez-Martin, and B. Carro. Adversarial environment reinforcement learning algorithm for intrusion detection. *Computer Networks*, 2019.

[22] E. Cavenaghi, G. Sottocornola, F. Stella, and M. Zanker. A systematic study on reproducibility of reinforcement learning in recommendation systems. *ACM Transactions on Recommender Systems*, 2023.

[23] S. C. Chan, S. Fishman, J. Canny, A. Korattikara, and S. Guadarrama. Measuring the reliability of reinforcement learning algorithms. *arXiv:1912.05663*, 2019.

[24] M. Chatterjee and Akbar-Siami Namin. Detecting phishing websites through deep reinforcement learning. In *IEEE annual computer software and applications conference*, 2019.

[25] J. Chen, S. Hu, H. Zheng, C. Xing, and G. Zhang. Gail-pt: An intelligent penetration testing framework with generative adversarial imitation learning. *Computers & Security*, 2023.

[26] S. Chen, N. Carlini, and D. Wagner. Stateful detection of black-box adversarial attacks. In *ACM Workshop on Security and Privacy on Artificial Intelligence*, 2020.

[27] X. Chen, Y. Nie, W. Guo, and X. Zhang. When llm meets drl: Advancing jailbreaking efficiency via drl-guided search. *Advances in Neural Information Processing Systems*, 2024.

[28] L. Chen et al. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 2021.

[29] K. L. Chung. Markov chains. *Springer-Verlag, New York*, 1967.

[30] C. Colas, O. Sigaud, and P. Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments. *CoRR*, 2018.

[31] J. Collyer, A. Andrew, and D. Hodges. ACD-G: Enhancing Autonomous Cyber Defense Agent Generalization Through Graph Embedded Network Representation. In *Workshop on Machine Learning for Cybersecurity ML4Cyber*, 2022.

[32] D. Corradini, Z. Montolli, M. Pasqua, and M. Ceccato. Deeprest: Automated test case generation for rest apis exploiting deep reinforcement learning. In *IEEE/ACM International Conference on Automated Software Engineering*, 2024.

[33] J. Cortellazzi et al. How to train your antivirus: Rl-based hardening through the problem-space. *arXiv:2402.19027*, 2024.

[34] W. Dabney, G. Ostrovski, D. Silver, and R. Munos. Implicit quantile networks for distributional reinforcement learning. *CoRR*, 2018.

[35] D. B. D'Ambrosio et al. Achieving Human Level Competitive Robot Table Tennis, 2024.

[36] R. De Silva, W. Guo, N. Ruaro, I. Grishchenko, C. Kruegel, and G. Vigna. {GuideEnricher}: Protecting the anonymity of ethereum mixing service users with deep reinforcement learning. In *USENIX Security Symposium*, 2024.

[37] X. Deng, M. Cen, M Jiang, and M. Lu. Ransomware early detection using deep reinforcement learning on portable executable header. *Cluster Computing*, 2024.

[38] R. R. Dos Santos, E. K. Viegas, A. O. Santin, and V. V. Cogo. Reinforcement learning for intrusion detection: More model longness and fewer updates. *IEEE Transactions on Network and Service Management*, 2022.

[39] G. Dulac-Arnold, D. Mankowitz, and T. Hester. Challenges of real-world reinforcement learning. *arXiv:1904.12901*, 2019.

[40] T. Eimer, M. Lindauer, and R. Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *International conference on machine learning*. PMLR, 2023.

[41] H. Emerson, L. Bates, C. Hicks, and V. Mavroudis. CybORG++: An Enhanced Gym for the Development of Autonomous Cyber Agents, 2024.

[42] J. Eom, S. Jeong, and T. Kwon. Fuzzing javascript interpreters with coverage-guided reinforcement learning for llm-based mutation. In *ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[43] L. Erdődi, Å Sommervoll, and F. M. Zennaro. Simulating sql injection vulnerability exploitation using q-learning reinforcement learning agents. *Journal of Information Security and Applications*, 2021.

[44] J. Evertz et al. Chasing shadows: Pitfalls in llm security research. *arXiv:2512.09549*, 2025.

[45] M. Faillon et al. How to better fit reinforcement learning for pentesting: A new hierarchical approach. In *European Symposium on Research in Computer Security*. Springer, 2024.

[46] Z. Fang, J. Wang, B. Li, S. Wu, Y. Zhou, and H. Huang. Evading anti-malware engines with deep reinforcement learning. *IEEE Access*, 2019.

[47] A. Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 2022.

[48] R. Feng, A. Hooda, N. Mangaokar, K. Fawaz, S. Jha, and A. Prakash. Stateful defenses for machine learning models are not yet secure against black-box attacks. In *ACM SIGSAC Conference on Computer and Communications Security*, 2023.

[49] M. Foley, C. Hicks, K. Highnam, and V. Mavroudis. Autonomous Network Defence Using Reinforcement Learning. In *ACM on Asia Conference on Computer and Communications Security*, ASIA CCS '22, 2022.

[50] M. Foley and S. Maffeis. HAXSS: Hierarchical Reinforcement Learning for XSS Payload Generation. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications*, 2022.

[51] M. Foley and S. Maffeis. Apirl: Deep reinforcement learning for rest api fuzzing. In *AAAI Conference on Artificial Intelligence*, 2025.

[52] M. Foley, M. Wang, Z. M, C. Hicks, and V. Mavroudis. Inroads into Autonomous Network Defence using Explained Reinforcement Learning. In *Conference on Applied Machine Learning in Information Security*, 2022.

[53] R. Gangupantulu, T. Cody, P. Park, A. Rahman, L. Eisenbeiser, D. Radke, R. Clark, and C. Redino. Using cyber terrain in reinforcement learning for penetration testing. In *IEEE International Conference on Omni-layer Intelligent Systems*, 2022.

[54] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell. Adversarial policies: Attacking deep reinforcement learning. *arXiv:1905.10615*, 2019.

[55] D. Goel, K. Moore, M. Guo, D. Wang, M. Kim, and S. Camtepe. Optimizing cyber defense in dynamic active directories through reinforcement learning. In *European Symposium on Research in Computer Security*. Springer, 2024.

[56] V. Gohil, H. Guo, S. Patnaik, and J. Rajendran. Attrition: Attacking static hardware trojan detection techniques using reinforcement learning. In *ACM SIGSAC conference on computer and communications security*, 2022.

[57] V. Gohil, S. Patnaik, D. Kalathil, and J. Rajendran. AttackGNN: Red-Teaming GNNs in hardware security using reinforcement learning. In *USENIX Security Symposium*, 2024.

[58] Y. Han, B. I. Rubinstein, T. Abraham, T. Alpcan, O. De Vel, S. Erfani, D. Hubczenko, C. Leckie, and

P. Montague. Reinforcement learning for autonomous defence in software-defined networking. In *International conference on decision and game theory for security*. Springer, 2018.

[59] M. Hausknecht and P. Stone. Deep recurrent q-learning for partially observable mdps. In *2015 aaai fall symposium series*, 2015.

[60] M. He, X. Wang, P. Wei, L. Yang, Y. Teng, and R. Lyu. Reinforcement learning meets network intrusion detection: A transferable and adaptable framework for anomaly behavior identification. *IEEE Transactions on Network and Service Management*, 2024.

[61] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. Deep reinforcement learning that matters. In *AAAI conference on artificial intelligence*, 2018.

[62] M. Hessel, H. van Hasselt, J. Modayil, and D. Silver. On inductive biases in deep reinforcement learning. *arXiv:1907.02908*, 2019.

[63] C. Hicks, V. Mavroudis, M. Foley, T. Davies, K. Highnam, and T. Watson. Canaries and Whistles: Resilient Drone Communication Networks with (or without) Deep Reinforcement Learning. In *ACM Workshop on Artificial Intelligence and Security*, AISec '23, 2023.

[64] Chris Hicks et al. Building better environments for autonomous cyber defence. *arXiv:2604.08805*, 2026.

[65] S. Hore, J. Ghadermazi, D. Paudel, A. Shah, T. Das, and N. Bastian. Deep packgen: A deep reinforcement learning framework for adversarial network packet generation. *ACM Transactions on Privacy and Security*, 2025.

[66] C. Hou, M. Zhou, Y. Ji, P. Daian, F. Tramer, G. Fanti, and A. Juels. Squirrl: Automating attack analysis on blockchain incentive mechanisms with deep reinforcement learning. *arXiv:1912.01798*, 2019.

[67] Y. Hsu and M. Matsuoka. A deep reinforcement learning approach for anomaly network intrusion detection system. In *IEEE international conference on cloud networking*, 2020.

[68] Z. Hu, R. Beuran, and Y. Tan. Automated penetration testing using deep reinforcement learning. In *IEEE European Symposium on Security and Privacy Workshops*, 2020.

[69] J. Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 2021.

[70] L. P. Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 1996.

[71] L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R. H Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, et al. Model-based reinforcement learning for atari. *arXiv:1903.00374*, 2019.

[72] Z. Kan, S. McFadden, D. Arp, F. Pendlebury, R. Jordaney, J. Kinder, F. Pierazzi, and L. Cavallaro. Tesseract: Eliminating experimental bias in malware classification across space and time (extended version). *arXiv:2402.01359*, 2024.

[73] S. Kim, S. Yoon, Jin-Hee Cho, D. S. Kim, Terrence J Moore, F. Free-Nelson, and H. Lim. Divergence: Deep reinforcement learning-based adaptive traffic inspection and moving target defense countermeasure framework. *IEEE Transactions on Network and Service Management*, 2022.

[74] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks. Evaluating Fuzz Testing. In *ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, 2018.

[75] A. Kvasov, M. Sahin, C. Hebert, and Anderson Santana De Oliveira. Simulating deception for web applications using reinforcement learning. In *European Symposium on Research in Computer Security*. Springer, 2023.

[76] M. Landen, K. Chung, M. Ike, S. Mackay, Jean-Paul Watson, and W. Lee. Dragon: Deep reinforcement learning for autonomous grid operation and attack detection. In *Annual Computer Security Applications Conference*, 2022.

[77] J R. Landis and G. G Koch. The measurement of observer agreement for categorical data. *biometrics*, 1977.

[78] P. Le Tolguenec et al. Exploration-driven reinforcement learning for avionic system fault detection (experience paper). In *ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[79] S. Lee, S. Wi, and S. Son. Link: Black-box detection of cross-site scripting vulnerabilities using reinforcement learning. In *ACM Web Conference*, 2022.

[80] Y. Lei et al. New challenges in reinforcement learning: a survey of security and privacy. *Artificial Intelligence Review*, 2023.

[81] Q. Li, M. Hu, H. H., M. Zhang, and Y. Li. Innes: An intelligent network penetration testing model based on deep reinforcement learning. *Applied Intelligence*, 2023.

[82] X. Li, X. Liu, L. Chen, R. Prajapati, and D. Wu. Alphaprog: reinforcement generation of valid programs for compiler fuzzing. In *AAAI Conference on Artificial Intelligence*, 2022.

[83] Z. Li, C. Huang, S. Deng, W. Qiu, and X. Gao. A soft actor-critic reinforcement learning algorithm for network intrusion detection. *Computers & Security*, 2023.

[84] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas. Application of deep reinforcement learning to intrusion detection for supervised problems. *Expert Systems with Applications*, 2020.

[85] M. Luo et al. Autocat: Reinforcement learning for automated exploration of cache-timing attacks. In *IEEE International Symposium on High-Performance Computer Architecture*, 2023.

[86] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 2018.

[87] R. Maeda and M. Mimura. Automating post-exploitation with deep reinforcement learning. *Computers & Security*, 2021.

[88] V. Mavroudis, G. Palmer, S. Farmer, K. S. Whitehead, D. Foster, A. Price, I. Miles, A. Caron, and S. Pasteris. Guidelines for applying rl and marl in cybersecurity applications. *arXiv:2503.04262*, 2025.

[89] S. McFadden, M. Foley, M. D'Onghia, C. Hicks, V. Mavroudis, N. Paoletti, and F. Pierazzi. Drmd: Deep reinforcement learning for malware detection under concept drift. In *Proc. of the AAAI Conference on Artificial Intelligence*, 2026.

[90] S. McFadden, M. Kan, L. Cavallaro, and F. Pierazzi. The impact of active learning on availability data poisoning for android malware classifiers. In *Annual Computer Security Applications Conference Workshops*, 2024.

[91] S. McFadden, Z. Kan, L. Cavallaro, and F. Pierazzi. Poster: Rpal-recovering malware classifiers from data poisoning using active learning. In *ACM SIGSAC Conference on Computer and Communications Security*, 2023.

[92] S. McFadden, M. Maugeri, C. Hicks, V. Mavroudis, and F. Pierazzi. Wendigo: Deep reinforcement learning for denial-of-service query discovery in graphql. In *IEEE Workshop on Deep Learning Security and Privacy*, 2024.

[93] L. Meng, R. Gorbet, and D. Kulić. Memory-based deep reinforcement learning for pomdps. In *IEEE/RSJ international conference on intelligent robots and systems*, 2021.

[94] A. Mirhoseini et al. A graph placement methodology for fast chipdesign. *Nature*, 2021.

[95] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. PmLR, 2016.

[96] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv:1312.5602*, 2013.

[97] S. Mohamed and R. Ejbali. Deep sarsa-based reinforcement learning approach for anomaly network intrusion detection system. *International Journal of Information Security*, 2023.

[98] A. Moore, A. Burke, M. Foley, A. Knack, C. Hicks, and V. Mavroudis. A fundamental research plan for autonomous cyber defence, 2025.

[99] T. T. Nguyen and Vijay Janapa Reddi. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems*, 2021.

[100] L. Nie, W. Sun, S. Wang, Z. Ning, J. J. Rodrigues, Y. Wu, and S. Li. Intrusion detection in green internet of things: a deep deterministic policy gradient-based algorithm. *IEEE Transactions on Green Communications and Networking*, 2021.

[101] E. Nikishin, M. Schwarzer, P. D'Oro, Pierre-Luc Bacon, and A. Courville. The Primacy Bias in Deep Reinforcement Learning. In *International Conference on Machine Learning*, 2022.

[102] J. Nyberg and P. Johnson. Training automated defense strategies using graph-based cyber attack simulations. *arXiv:2304.11084*, 2023.

[103] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped DQN. *CoRR*, 2016.

[104] A. Patterson, S. Neumann, M. White, and A. White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 2024.

[105] B. Paudel and G. Amariucai. Reinforcement learning approach to generate zero-dynamics attacks on control systems without state space models. In *European Symposium on Research in Computer Security*. Springer, 2023.

[106] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. In *USENIX Security Symposium*, 2019.

[107] T. V. Phan and T. Bauschert. Deepair: Deep reinforcement learning for adaptive intrusion response in software-defined networks. *IEEE Transactions on Network and Service Management*, 2022.

[108] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro. Intriguing properties of adversarial ml attacks in the problem space. In *IEEE symposium on security and privacy*, 2020.

[109] V. Praveena, A V., P Chinnasamy, I. Ali, R. Alroobaea, S. Y. Alyahyan, and Muhammad Ahsan Raza. Optimal deep reinforcement learning for intrusion detection in uavs. *Computers, Materials & Continua*, 2022.

[110] R. H. Randhawa, N. Aslam, M. Alauthman, M. Khalid, and H. Rafiq. Deep reinforcement learning based evasion generative adversarial network for botnet detection. *Future Generation Computer Systems*, 2024.

[111] A. Rashid and J. Such. Malprotect: Stateful defense against adversarial query attacks in ml-based malware detection. *IEEE Transactions on Information Forensics and Security*, 2023.

[112] M. Rigaki and S. Garcia. The power of meme: Adversarial malware creation with model-based reinforcement learning. In *European Symposium on Research in Computer Security*. Springer, 2023.

[113] A. Romdhana, A. Merlo, M. Ceccato, and P. Tonella. Deep reinforcement learning for black-box testing of android apps. *ACM Transactions on Software Engineering and Methodology*, 2022.

[114] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N.lai Bissantz, M. Muench, and T. Holz. SoK: Prudent Evaluation Practices for Fuzzing. In *IEEE Symposium on Security and Privacy*, 2024.

[115] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*. PMLR, 2015.

[116] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[117] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, 2017.

[118] K. Sethi, R. Kumar, N. Prajapati, and P. Bera. Deep reinforcement learning based intrusion detection system for cloud infrastructure. In *International Conference on COMmunication Systems & NETworkS*, 2020.

[119] A. Sharma and M. Singh. Batch reinforcement learning approach using recursive feature elimination for network intrusion detection. *Engineering Applications of Artificial Intelligence*, 2024.

[120] E. Shereen, D. Ristea, S. McFadden, B. Hasircioglu, V. Mavroudis, and C. Hicks. One pic is all it takes: Poisoning visual document retrieval augmented generation with a single image. *arXiv:2504.02132*, 2025.

[121] R. Simon, P. Libin, and W. Mees. Learning Robust Penetration-Testing Policies under Partial Observability: A systematic evaluation, 2025. arXiv:2509.20008.

[122] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger. Defining and Characterizing Reward Gaming. *Advances in Neural Information Processing Systems*, 2022.

[123] R. Sommer and V. Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *IEEE symposium on security and privacy*, 2010.

[124] M. Standen, D. Bowman, O. Naish, et al. Cyber operations research gym. https://github.com/cage-challenge/CybORG, 2022.

[125] J. Su et al. Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing. In *IEEE/ACM International Conference on Automated Software Engineering*, 2022.

[126] J. Sun, T. Zhang, X. Xie, L. Ma, Y. Zheng, K. Chen, and Y.g Liu. Stealthy and efficient adversarial attacks against deep reinforcement learning. In *AAAI conference on artificial intelligence*, 2020.

[127] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. 2018.

[128] F. Terranova, A. Lahmadi, and I. Chrisment. Leveraging deep reinforcement learning for cyber-attack paths prediction: Formulation, generalization, and evaluation. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2024.

[129] S. Tharewal, M. W. Ashfaque, S. S. Banu, P. Uma, S. M. Hassen, and M. Shabaz. Intrusion detection system for industrial internet of things based on deep reinforcement learning. *Wireless Communications and Mobile Computing*, 2022.

[130] L. Tong, A. Laszka, C. Yan, N. Zhang, and Y. Vorobeychik. Finding needles in a moving haystack: Prioritizing alerts with adversarial reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2020.

[131] I. Tsingenopoulos, D. Preuveneers, L. Desmet, and W. Joosen. Captcha me if you can: Imitation games with reinforcement learning. In *IEEE European Symposium on Security and Privacy*, 2022.

[132] I. Tsingenopoulos et al. How to train your antivirus: Rl-based hardening through the problem space. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2024.

[133] I. Tsingenopoulos et al. The adaptive arms race: Redefining robustness in ai security. Research in Attacks, Intrusions and Defenses, 2025.

[134] H. Van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *AAAI conference on artificial intelligence*, 2016.

[135] S. Vyas, V. Mavroudis, and P. Burnap. Towards the deployment of realistic autonomous cyber network defence: A systematic review. *ACM Computing Surveys*, 2025.

[136] J. Wang, L. Qixu, W. Di, Y. Dong, and X. Cui. Crafting adversarial example to bypass flow-&ml-based botnet detector via rl. In *international symposium on research in attacks, intrusions and defenses*, 2021.

[137] Z. Wang, H. He, Z. Wan, and Y. Sun. Coordinated topology attacks in smart grid using deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 2020.

[138] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*. PMLR, 2016.

[139] C. J. Watkins and P. Dayan. Q-learning. *Machine learning*, 1992.

[140] D. Wu, B. Fang, J. Wang, Q. Liu, and X. Cui. Evading machine learning botnet detection models via deep reinforcement learning. In *IEEE International Conference on Communications*, 2019.

[141] X. Wu, W. Guo, H. Wei, and X. Xing. Adversarial policy training against deep reinforcement learning. In *USENIX Security Symposium*, 2021.

[142] P. R. Wurman et al. Outracing champion Gran Turismo drivers with deep reinforcement learning. *Nature*, 2022.

[143] X. Xu, Y. Sun, and Z. Huang. Defending ddos attacks using hidden markov models and cooperative reinforcement learning. In *Pacific-Asia Workshop on Intelligence and Security Informatics*. Springer, 2007.

[144] C. Yang, A. Kortylewski, C. Xie, Y. Cao, and A. Yuille. Patchattack: A black-box texture-based attack with reinforcement learning. In *European Conference on Computer Vision*. Springer, 2020.

[145] Y. Yang et al. Behaviour-diverse automatic penetration testing: a coverage-based deep reinforcement learning approach. *Frontiers of Computer Science*, 2025.

[146] S. Yu et al. Deep q-network-based open-set intrusion detection solution for industrial internet of things. *IEEE Internet of Things Journal*, 2023.

[147] D. Zhan et al. Psp-mal: Evading malware detection via prioritized experience-based reinforcement learning with shapley prior. In *Annual Computer Security Applications Conference*, 2023.

[148] W. Zhang, D. Zhou, L. Li, and Q. Gu. Neural thompson sampling. *arXiv:2010.00827*, 2020.

[149] T. Zhang et al. When moving target defense meets attack prediction in digital twins: A convolutional and hierarchical reinforcement learning approach. *IEEE Journal on Selected Areas in Communications*, 2023.

[150] W. Zhao, J. P. Queralta, and T. Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *IEEE Symposium Series on Computational Intelligence*, 2020.

[151] K. Zhao et al. Structural attack against graph based android malware detection. In *ACM SIGSAC conference on computer and communications security*, 2021.

[152] D. Zhou, L. Li, and Q. Gu. Neural contextual bandits with ucb-based exploration. In *International conference on machine learning*. PMLR, 2020.

[153] A. Zou et al. Universal and transferable adversarial attacks on aligned language models. *arXiv:2307.15043*, 2023.